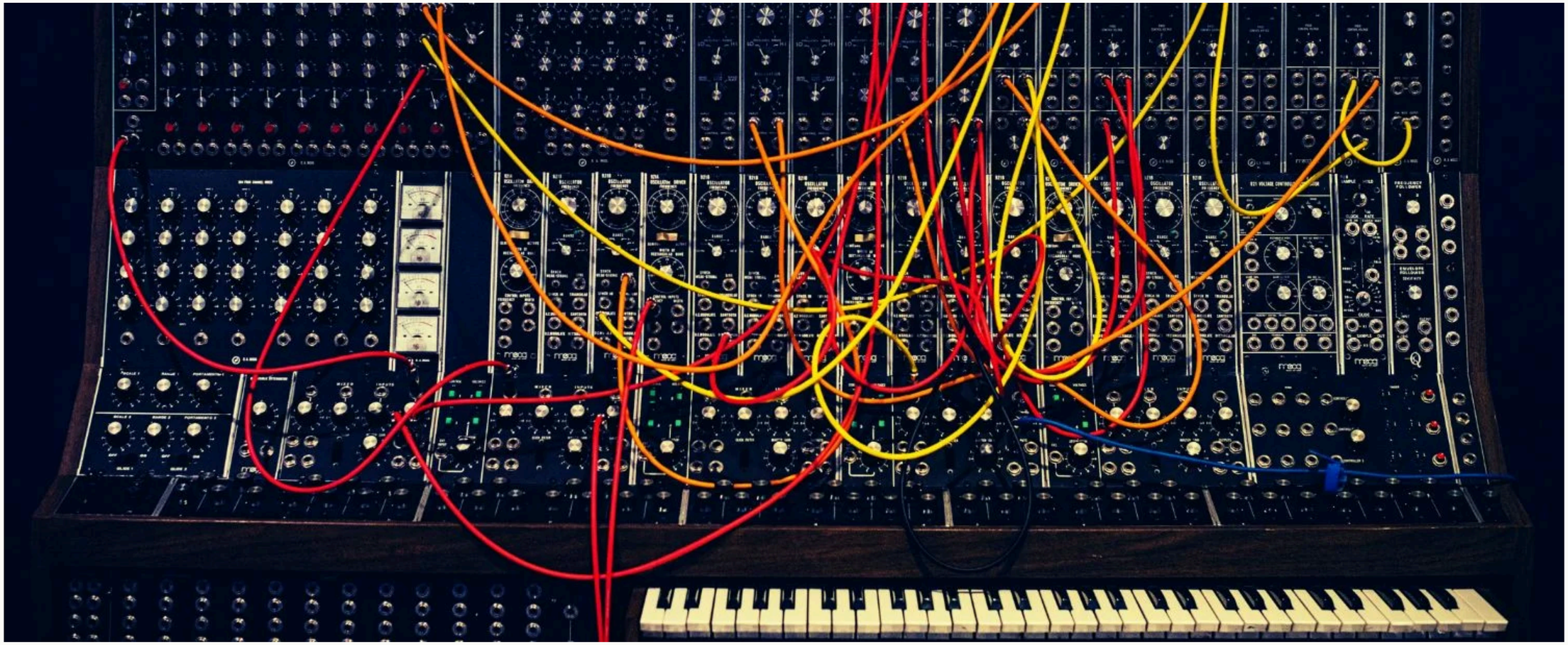
LLM COURSE · PROJECT PROPOSAL

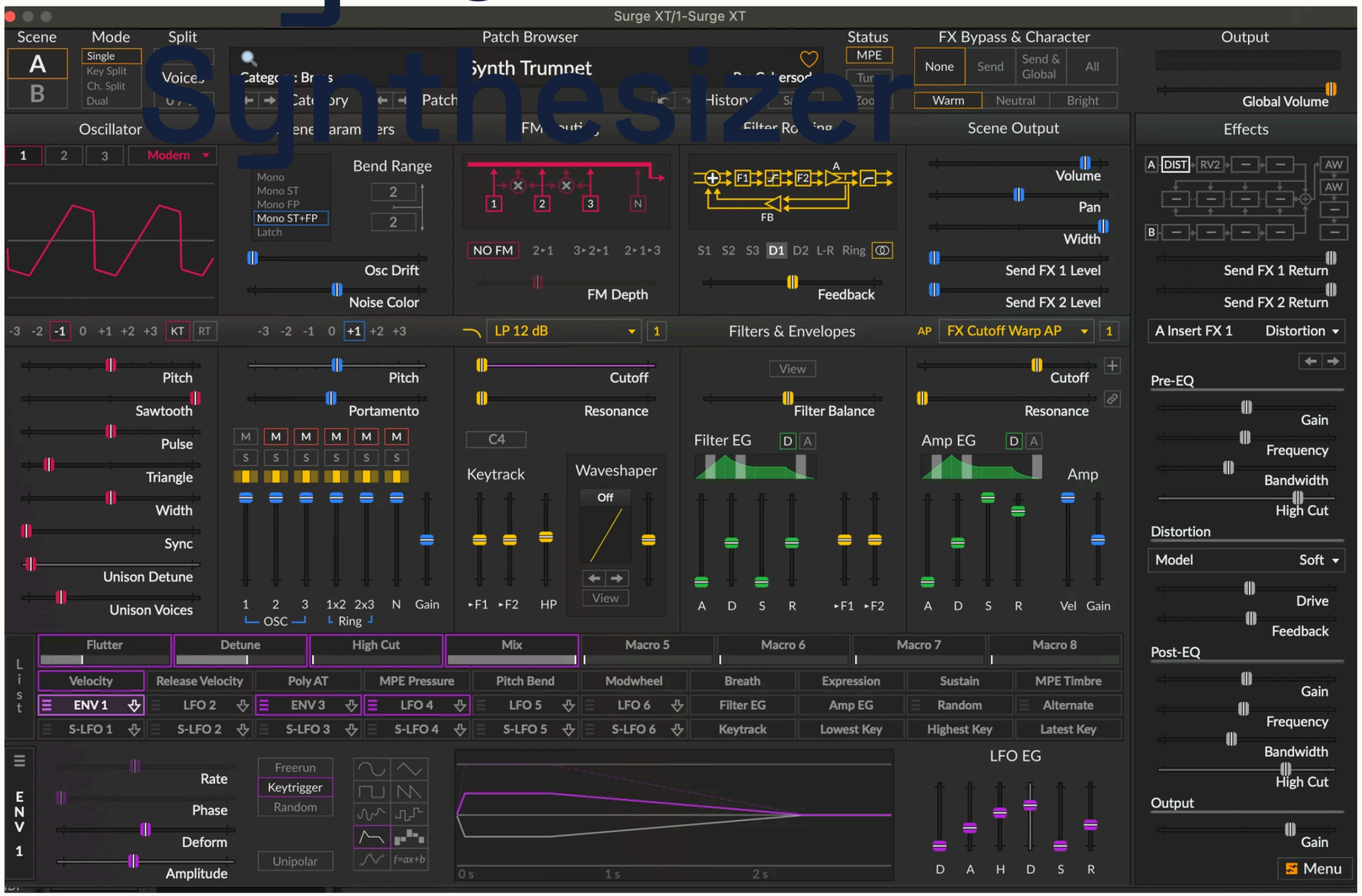
SynthTeach

A sound-design teacher for beginners that reads your synth patch and your words, then changes the right knobs and explains why.

Synthesizer



Digital Synthesizer



HOW PEOPLE ACTUALLY DESCRIBE SOUND

bubbly shimmery muddy muffled dense raspy small nasal sweet snappy sparkly
vintage tight dusty sharp veiled clangy squishy bell-like fuzzy plucky bitey organic spiky
tinny velvety dry woolly boxy bouncy round thick dull nostalgic chewy sizzly smooth huge
pulsing cold honky squelchy airy upfront meaty massive polished thin stiff glassy
boingy creamy woody harsh rubbery glossy alive “more sun in it” bright
“make it breathe” crispy in your face grimy pillowy clean fat screamy icy silky plasticky
“needs more air” lush gritty juicy sterile haunting nasty buzzy shrill analog filthy swirly
floaty beefy buttery “like a cheap toy” static full washy wet crunchy punchy
distant phat wobbly breathy hollow digital warm cinematic aggressive
spacey chunky dreamy metallic edgy fizzy dark boomy wide “like vinyl” tiny
growly “less like a robot” 80s mellow laser-y soft lo-fi piercing shiny
“too much in my face” under a blanket clicky sparse retro gnarly dirty

Patch and request in, changes and explanation out

INPUT

“It sounds too muddy, make it bright and snappy”

```
<a_filter1_cutoff value="-30.0" />
<a_filter1_resonance value="0.10" />
<a_filter1_envmod value="10.0" />
<a_env1_attack value="-3.0" />
<a_env1_decay value="0.0" />
<a_env1_sustain value="1.00" />
<a_env1_release value="-2.0" />
<a_env2_decay value="-1.0" />
```



Opening the filter clears the mud and adds brightness. A fast attack, short decay and no sustain make it snappy.

```
- <a_filter1_cutoff value="-30.0" />
+ <a_filter1_cutoff value="-6.0" />
- <a_filter1_envmod value="10.0" />
+ <a_filter1_envmod value="30.0" />
- <a_env1_attack value="-3.0" />
+ <a_env1_attack value="-8.0" />
- <a_env1_decay value="0.0" />
+ <a_env1_decay value="-2.5" />
- <a_env1_sustain value="1.00" />
+ <a_env1_sustain value="0.00" />
- <a_env2_decay value="-1.0" />
+ <a_env2_decay value="-3.0" />
```

Beginners can hear what they want, but not which knob gets them there

- **Who:** beginners to intermediate and hobby producers who can say “warmer” or “wider” but face hundreds of parameters.
- **Beginners describe the effect, not the knob: “make it plucky”, not “shorten the decay”.**
- **Exists:** presets, tutorials, forums such as r/synthrecipes, and paid courses.
- **The Gap:** presets don’t teach, and tutorials and forums don’t know your current patch. A large chatbot can work from a pasted patch, *but how well it understands beginners is untested* (besides being slow and costly).
- **Value: the user gets the change and learns which knob made it.**

The task

Given a patch and a beginner's plain-language request, produce the parameter changes and a short explanation, changing only what the request needs.

- **Editing, not creating.**
- **Input:** one sentence, plus a fixed list of the patch's key settings.
- **Output:** changed parameters + what changed and what to listen for.
- **State matters:** "brighter" means something different when the filter is already open. (Maybe adds octaves? shimmer?)

Closest prior work, most similar first

No combination of: small text-only model · synth patch editing · real human requests · teaching

System	What it does	What it leaves open
Preset Mutator (free browser tool)	Edits a Vital preset along five sliders, such as darker/brighter and cleaner/dirtier	No language input.
synth-rl (early GitHub project)	Edits presets of the Vital synth from text such as “brighter”	Reinforcement Learning, Large audio-capable models that listen in a loop (Gemini, or 3–30B local). No beginner language, no human test, not built to teach
InstructFX2FX (2026)	Step-by-step text edits of existing EQ and reverb settings	Not for synthesizers, only EQ and reverb. Large LLM plus audio matching, single-word tests
AutoSynth (2026)	Generates a whole synth patch from audio or text	Doesn’t edit your patch, doesn’t teach. Starts from scratch
CTAG (MIT, 2024)	Searches synth settings until the sound matches a text, such as “a dog barking”	From scratch only, slow search, sound effects
SynthScribe (2023)	Smart search	No editing
LLM2Fx (Sony, 2025)	Sets effects	Ignores state

What’s new:

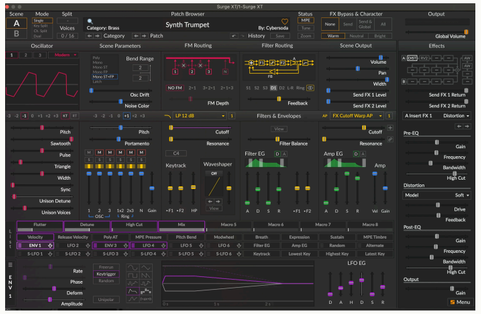
1. **Context-based editing via LLM:** starts from your patch and your words, and changes only what’s needed.
2. **Built for free language:** trained and tested on how beginners or semi-pro actually describe sound.
3. **Teaching:** every edit comes with what changed and what to listen for.
4. **Small and measured:** a small fine-tuned model, tested with a new dataset of real beginner language and a listening protocol. Can it happen in real time?

Each piece exists somewhere. None of the systems I found combines them.

The pipeline

Surge XT patch → Simplified patch to text → Pair with request → **Small fine-tuned model** → Validate result → Apply to a copy

- **Surge XT:** a free, open-source synth. Each patch is an XML file with about 570 settings.
- **Patch to text:** code keeps ~30 key settings (filter, envelopes, oscillators, drive) as “name = value” lines.
- **Pair with request:** the user’s own words are added, e.g. “make it warmer”.
- **Small fine-tuned model:** writes the new values and a short explanation.
- **Validate result:** every name must exist and every value must be in range.
- **Apply to a copy:** the new values go into a copy of the patch, which opens in Surge XT. The original stays untouched.



Model description



- **Recipe from the closest prior work:** for words → EQ settings, a small instruction-tuned LLM with LoRA had the best median score (One Prompt, Many Sounds, 2026).
- **Model:** Phi-3.5-mini (3.8B), as in that paper, or a smaller Qwen (0.5–1.5B) if speed matters more.
- **LoRA:** the model stays frozen and only small add-on weights (~1%) are trained, so training stays cheap. Done with Hugging Face’s peft library.

Train Set



Naive option: Fully generated by LLM

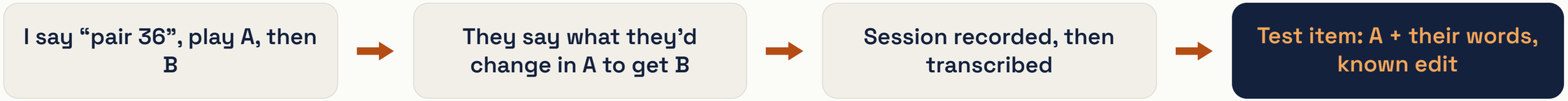
Starting point:

- **Existing data:** 3,008 built-in patches, which I'll split into train, validation and test.
- **Rules**: about 15 trivial edit rules, plus 15 non-trivial and combinations of rules (as a start), in five families (oscillator, filter, envelope, modulation, drive), from community advice (r/synthrecipes, etc.).
- Each rule is bound to a description word.
- **Example rule:** “muddy” → cutoff up 25% of its range, only if it’s below 70%; otherwise drive up.
- **Prompt creation:**
- **Each example:** patch + request. target = the edit (rule applied on patch) + explanation.

GPT or Claude (API) writes the request, varying the phrasing in different styles.

- **Size:** ~3,000 training and ~300 validation examples, from different patches and prompts.
- **Quality:** automatic: code renders every edit, measuring audio features, and drops edits that didn’t change the sound. By hand: I read 100 requests and listen to ~50 edits.

Test set generation



- **People:** 5–10 beginners to intermediate, about 20 minutes each (15–20 pairs).
- **Pairs:** **B** patch is **A** after one or two known rules, so the right answer is known.
- **One question, spoken:** “What would you change in A to get to B?” The answer is the model’s input.
- **No software:** the whole session is recorded. Whisper transcribes it, an LLM splits it into pair → answer.
- **Feature check:** an LLM tags each answer with a feature and direction (“muddier” → brightness down).
- **Size:** ~150 items. About 5 pairs are heard by everyone, to measure how much people agree.

How success is measured

- **Main score, audio features direction:** the model's sound must move the same feature as the known edit, in the same direction, by a similar amount.
- **The features:** measured on the actual signal with librosa. Brightness = spectral centroid, attack = 10–90% rise time, width = stereo side-to-mid ratio, movement = variation of brightness and level over time, dirt = spectral flatness.
- **Second score, knob match:** same parameter, same direction as the known edit.
- **Nothing else changed:** the other features stay within a set tolerance. An invalid edit (unknown parameter, out-of-range value) counts as a failure, and the original patch is kept.
- **State test:** the same model with and without the patch as input. If the scores match, the state didn't matter.
- **Human validation:** on the pairs people heard, a feature must move when they name it ("brighter" = centroid up). I also listen to about 30 outputs with the labels hidden.
- **Baselines:** keyword rules, a small classifier that only picks the edit type, and a zero-shot and few-shot large LLM, all given the same input as the fine-tuned model. Results are reported by family: oscillator, filter, envelope, modulation, drive.

What could go wrong, and the plan for it

- **Audio features do not represent actual change.**
- **Vague words (“Not enough sun in it”):** measure agreement on the shared pairs and report high and low agreement separately.
- **Many changes give the same sound:** score the rendered audio, not only the parameters.
- **Synthetic requests may not sound like beginners:** test on human wording the model never saw.
- Is it just an easy **intent detection** problem, since there are only 30 rules?